

CS1001: Introduction to Computing

Module 1 and 2: Notes by Sudeep K S, August 2026

Set 1: Number Representations

- Representing numbers: The 'normal' way of doing it is **Decimal** system.

Has 10 different symbols to represent a number: 0, 1, 2, 3, ... 9.

A number written as 928 means $9 \times 100 + 2 \times 10 + 8 \times 1$
Or, $9 \times 10^2 + 2 \times 10^1 + 8 \times 1$

The symbols in each position have a different meaning (or different value).

If we have n different symbols, we can apply the same technique and represent numbers in a similar way. 0 to $(n - 1)$ will use the n different symbols, and from the number n onwards, we use the position to get a bigger number.

In computers, it is easy to deal with just two symbols: 0 and 1 (say, on and off). So, we use a 'binary' system, that uses only these two symbols.

- A different way, same concept: **Binary** system

First two numbers same: 0 means 0, 1 means 1.

It stops there, since we have only 2 symbols. For 2, we use 10. It means $1 \times 2 + 0 \times 1$.

3 is 11. Then comes 100, it means 4. (That is $1 \times 4 + 0 \times 2 + 0 \times 1$).

Here, position values are 1, 2, 4, 8 and so on. Or, 1 (or 2^0), 2^1 , 2^2 , 2^3 , 2^4 , ... (as we go left). So 100 has 1 in 4s' position, 0 in 2s' position, 0 in ones' position.

Similarly, 11011 means $1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 1 = 27$.

We can represent any number like this. With n bits, we can represent up to $2^n - 1$.

In binary system, one symbol (0 or 1) is called a bit. (8 of them together is called a byte).

- There are other systems also, using 8 symbols (**octal**) or 16 symbols (**hexadecimal**).

Converting a number to binary

We can convert a number given to binary by repeatedly dividing it by 2 and writing down the remainder, starting from the right end (starting from least significant bits).

Here is a small example, take the number twenty-seven.

$$27 / 2 = 13, \text{ remainder: } 1$$

This 1 goes in the 1s' position.

$$13 / 2 = 6, \text{ remainder: } 1$$

This 1 goes in 2s' position.

$$6 / 2 = 3, \text{ remainder: } 0$$

This 0 goes in 4s' position.

$$3 / 2 = 1, \text{ remainder: } 1$$

This 1 goes in the 8s' position.

$$1 / 2 = 0, \text{ remainder: } 1$$

This 1 goes in 16s' position.

So we get 11011, which is the binary representation of twenty seven (decimal: 27).

$$27 = 16 + 8 + 2 + 1.$$

Conversion to other systems: Octal or Hexadecimal

We can convert a number first to binary and then group it into groups of 3 bits each (starting from the rightmost end), to get the octal representation.

Using this technique, we can see that the number given above (27), 11011 in binary, becomes 011-011. which is 33. It is $3 \times 8 + 3 \times 1$.

In hexadecimal system, we use 16 symbols. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A (means 10), B (11), C (12), D (13), E (14) and F (15). For example, 3DF means $3 \times 16^2 + 13 \times 16 + 15 \times 1$.

To convert a number to hexadecimal, we can first convert it to binary and then group the bits into groups of 4 each, this also starting from the right end. 27 (11011) now becomes 1-1011. That is 1 and then 11, which is 1B. $27 = 1 \times 16 + 11 \text{ (B)} \times 1$.

Binary arithmetic

Addition is similar to adding two or more decimal numbers. Carry goes to one position left.

$$\begin{array}{r} 1\ 0\ 0\ 1\ + \\ 1\ 0\ 1\ \\ \hline 1\ 1\ 1\ 0\ \\ \\ 1\ 1\ 1\ + \\ 1\ \\ \hline 1\ 0\ 0\ 0\ \end{array}$$

Representation of negative numbers

There are different ways to represent negative numbers when we are using the binary system. One simple way is to use one bit for the sign of the number.

If we have 4 bits to represent a number, use the first bit as the sign and remaining bits as value.

That is, 0101 means 5 and 1101 means -5.

If we use this system, we can represent numbers -7 (1111) to 7 (0111) using 4 bits.

If we had 8 bits, it goes from -127 (11111111) to 127 (01111111).

But the catch is that it has two 0s: 1000 and 0000 both refer to the same number.

So we get only 15 different numbers (is we had four bits), though there are 16 different symbol sequences using 4 bits.

A way to fix this is using 2s' complement if the number when the number is negative.

Here also, the first bit (0 or 1) tells whether the number is negative (1) or not (0).

For negative numbers, use the 2s' complement.

Here is how to compute 2s' complement:

Complement every bit (switch 0 to 1 and 1 to 0); Then add 1.

For example, 0101 (5) becomes $1010 + 1 = 1011$. So, this is -5.

Similarly,

1: 0001 becomes $1110 + 1 = 1111$. That means -1 is represented as 1111.

2: 0010 becomes $1101 + 1 = 1110$. (It is -2)

3: 0011 becomes $1100 + 1 = 1101$. (It is -3)

4: 0100 becomes $1011 + 1 = 1100$. (It is -4)

5: 0101 becomes 1011. (it is shown above)

6: 0110 becomes $1001 + 1 = 1010$. (It is -6)

7: 0111 becomes $1000 + 1 = 1001$. (This is -7)

Using this technique, we can represent from -8 (1000) to +7 (0111) with the same 4 bits.

Now we are utilizing all the 16 sequences, they refer to 16 different numbers.

If we had n bits to represent an integer, now we can go from -2^{n-1} to $2^{n-1} - 1$.

In C, an integer is of 4 bytes (32 bits) in most of the new systems.

It means it can hold numbers from -2^{31} to $2^{31} - 1$.

Getting the number if we are given 2s' complement

It is easy to get back the number from its 2s' complement. Just take 2s' complement once again.

Flip 0s to 1s and 1s to 0s, then add 1. You get the original number back. (Verify it for yourself.)

Representing real (floating-point) numbers

- A 'float' datatype variable is stored in a slightly different way, as seen in the picture below.



Here we have one bit exclusively for sign, 8 bits for exponent and 23 bits for the 'mantissa' (or the value). From these, we get the number as

$$\text{Number} = (1 + \text{Mantissa part considered as a fraction}) \times \text{Base}^{\text{Exponent}}$$

Here, Base is 2.

The 'Exponent' field contains the exponent + 127 (that is the standard used, so that 8 bits can easily account for negative exponents also). Smallest number: 1. Range: -126 to +127.

Mantissa part is like numbers after the decimal (binary) point: The positions are $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, ...

For example, if Mantissa is 011, the number to be multiplied is 1.011 (that is $1 + \frac{1}{4} + \frac{1}{8}$); and if the exponent bits show 01111100 (124), then the actual exponent is -3; and the number is

$$(1 + \frac{1}{4} + \frac{1}{8}) \times 2^{-3}$$

If the exponent is positive, move the decimal point to right by that many positions (add 0s if needed to the right). If negative, move to left. If exponent is 0, no decimal point, no shift.

Range of 'float' given like this:

Negative numbers: -3.4×10^{38} to -1.17×10^{-38}

Positive numbers: 1.17×10^{-38} to 3.4×10^{38}

'double' is similar but uses more bits for exponent as well as for mantissa.